

Jupyter Notebook 入门教程

Print to PDF

Contents

- 6.1. 什么是 Jupyter Notebook?
- 6.2. Notebook 的核心：单元格 (Cells)
- 6.3. 基本工作流程
- 6.4. 魔法命令 (Magic Commands)
- 6.5. 总结

6.1. 什么是 Jupyter Notebook?

Jupyter Notebook 是一个开源的 Web 应用程序，允许您创建和共享包含实时代码、方程式、可视化和叙述性文本的文档。它的名字来源于其支持的三种核心编程语言：**Julia**、**Python** 和 **R**。

对于科学计算和数据分析领域，Jupyter Notebook 是一个不可或缺的工具，因为它提供了一个交互式的环境，让我们可以逐步编写、测试和展示我们的代码和分析结果。

6.2. Notebook 的核心：单元格 (Cells)

Jupyter Notebook 的文档由一系列 **单元格** 组成。每个单元格都是一个独立的代码块或文本块。主要有两种类型的单元格：

1. **代码单元格 (Code Cell)**: 用于编写和执行代码。
2. **Markdown 单元格 (Markdown Cell)**: 用于编写格式化的文本、标题、列表、图片和数学公式。

您可以通过工具栏上的下拉菜单来切换当前选中单元格的类型。

6.2.1. 代码单元格示例

代码单元格是您编写和运行 Python 代码的地方。要执行一个代码单元格，您可以选中它，然后点击工具栏上的“运行” (Run) 按钮，或者使用快捷键 **Shift + Enter**。

执行后，代码的输出会直接显示在该单元格的下方。

```
# 这是一个代码单元格
# 让我们执行一些简单的 Python 代码
a = 5
b = 10
result = a + b

# 单元格的最后一个表达式的值会自动输出
result
```

15

```
# 您也可以使用 print() 函数来输出信息
print(f"变量 a 的值是: {a}")
print(f"变量 b 的值是: {b}")
print(f"它们的和是: {result}")
```

变量 a 的值是: 5
变量 b 的值是: 10
它们的和是: 15

重要概念：内核 (Kernel)

当您运行代码时，实际上是由一个在后台运行的 **内核 (Kernel)** 来执行的。这个内核会记住您之前执行过的所有代码的状态，比如定义的变量 `a`、`b`，和 `result`。这意味着您可以在一个单元格中定义变量，然后在后续的单元格中使用它们。

如果您的代码卡住或者想重新开始，可以从菜单栏的“内核” (Kernel) 选项中选择“重启” (Restart) 或“重启并清空输出” (Restart & Clear Output)。

6.2.2. Markdown 单元格示例

Markdown 是一种轻量级的标记语言，可以用来编写格式优美的文本。这使得您可以在代码旁边添加详细的解释和文档。

双击这个单元格可以查看其 Markdown 源代码。再次运行它 (**Shift + Enter**) 则会渲染成格式化的文本。

下面是一些常用的 Markdown 语法：

```
# 一级标题
## 二级标题
### 三级标题

**这是粗体文本**

*这是斜体文本*

- 这是一个无序列表项 1
- 这是一个无序列表项 2

1. 这是一个有序列表项 1
2. 这是一个有序列表项 2
```

您甚至可以使用 LaTeX 语法编写精美的数学公式，例如爱因斯坦的质能方程： $E = mc^2$ 。

或者更复杂的，薛定谔方程：

```
$$
i\hbar\frac{\partial}{\partial t}\Psi(\mathbf{r},t) = \left[-\frac{\hbar^2}{2m}\nabla^2 + V(\mathbf{r},t)\right]\Psi(\mathbf{r},t)
$$
```

6.3. 基本工作流程

- **创建新单元格:** 点击工具栏上的 `+` 按钮。
- **运行单元格:** 选中单元格后，按 `Shift + Enter`（运行并移动到下一个单元格）或 `Ctrl + Enter`（运行并停留在当前单元格）。
- **切换单元格类型:** 使用工具栏的下拉菜单，或在命令模式下（按 `Esc` 键进入）按 `M` 切换到 Markdown，按 `Y` 切换到代码。
- **保存 Notebook:** 按 `Ctrl + S` 或点击工具栏的保存图标。

6.4. 魔法命令 (Magic Commands)

Jupyter Notebook 还提供了一些特殊的“魔法命令”，它们以 `%` 或 `%%` 开头，可以提供一些便捷的功能。

- **行魔法命令 (`%`):** 只作用于单行。
- **单元格魔法命令 (`%%`):** 作用于整个单元格。

```
# %timeit: 一个行魔法命令，用于测量单行代码的执行时间
%timeit sum(range(1000))
```

7.16 μ s $\pm$ 133 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

```
%%timeit

# %%timeit: 一个单元格魔法命令，用于测量整个单元格代码的执行时间
# 注意: %%timeit 必须是单元格的第一行
total = 0
for i in range(1000):
    total += i
```

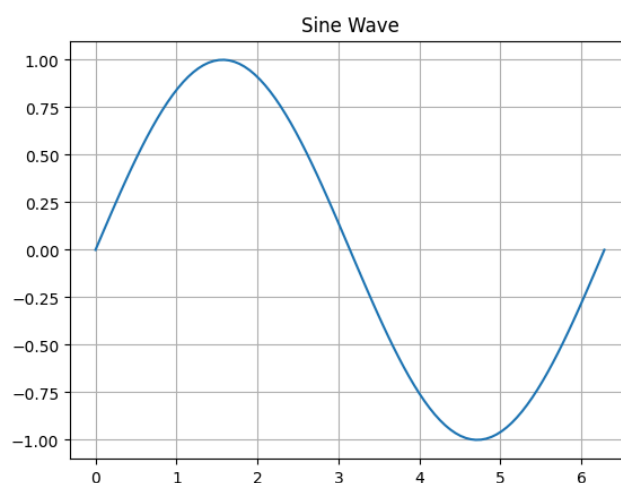
27.4 μ s $\pm$ 3.3 μ s per loop (mean $\pm$ std. dev. of 7 runs, 10,000 loops each)

另一个非常有用的魔法命令是 `%matplotlib inline`，它允许 Matplotlib 绘制的图形直接嵌入到 Notebook 中。不过，在许多现代 Jupyter 环境中，这通常是默认行为，无需显式声明。

```
# %matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)

plt.plot(x, y)
plt.title("Sine Wave")
plt.grid(True)
plt.show()
```



../_images/7de311c3f01ec775c191577bed57eba17710eddb40d7e33a9bb89027aef970a8.png

6.5. 总结

Jupyter Notebook 是一个强大的工具，它将代码、输出、文本和可视化结合在一起，非常适合用于探索性数据分析、教学演示和科学研究。希望这个简短的教程能帮助您入门！

本讲义的每个页面，右上角都有一个下载按钮，可以将该页面作为 Jupyter Notebook 文件下载下来，方便您在本地进行实验和学习。